EXPERIMENT-23

23. C Program for Encryption and Decryption Using S-DES in Counter (CTR) Mode

AIM

To write a C program to encrypt and decrypt binary plaintext using Simplified DES (S-DES) in Counter (CTR) mode, with the key `0111111101` and the counter starting at `00000000`.

PROCEDURE

1. Initialize the 10-bit key as `0111111101` and the initial counter as `00000000`.
2. Generate the two S-DES subkeys using the key-generation process.
3. Divide the plaintext into 8-bit blocks: `00000001`, `00000010`, and `00000100`.
4. Encrypt each counter value using S-DES to generate a keystream block.
5. XOR each keystream block with the corresponding plaintext block to produce ciphertext.
6. For decryption, generate the same keystream and XOR it with the ciphertext to recover the original plaintext.

CODE (C)

```
#include <stdio.h>

int P10[] = {3,5,2,7,4,10,1,9,8,6};
int P8[]  = {6,3,7,4,8,5,10,9};
int IP[]  = {2,6,3,1,4,8,5,7};
int IPI[] = {4,1,3,5,7,2,8,6};
int EP[]  = {4,1,2,3,2,3,4,1};
int P4[]  = {2,4,3,1};

int S0[4][4] = {
    {1,0,3,2}, {3,2,1,0},
    {0,2,1,3}, {3,1,3,2}
};

int S1[4][4] = {
    {0,1,2,3}, {2,0,1,3},
```

```
    {3,0,1,0}, {2,1,0,3}
};

int K1, K2;

int permute(int x, int p[], int n, int bits) {
    int r = 0, i;
    for (i = 0; i < n; i++)
        r = (r << 1) | ((x >> (bits - p[i])) & 1);
    return r;
}

int shift5(int x, int n) {
    int l = (x >> 5) & 31, r = x & 31;
    l = ((l << n) | (l >> (5 - n))) & 31;
    r = ((r << n) | (r >> (5 - n))) & 31;
    return (l << 5) | r;
}

void keygen(int key) {
    int t = permute(key, P10, 10, 10);
    t = shift5(t, 1);
    K1 = permute(t, P8, 8, 10);
    t = shift5(t, 2);
    K2 = permute(t, P8, 8, 10);
}

int f(int b, int key) {
    int l = b >> 4, r = b & 15;
    int t = permute(r, EP, 8, 4) ^ key;
    int a = (t >> 4) & 15, c = t & 15;
    int row, col, s;

    row = ((a & 8) >> 2) | (a & 1);
    col = (a >> 1) & 3;
    s = S0[row][col] << 2;

    row = ((c & 8) >> 2) | (c & 1);
    col = (c >> 1) & 3;
    s |= S1[row][col];

    return ((l ^ permute(s, P4, 4, 4)) << 4) | r;
}

int encrypt(int x) {
    x = permute(x, IP, 8, 8);
    x = f(x, K1);
    x = ((x & 15) << 4) | (x >> 4);
    x = f(x, K2);
```

```
    return permute(x, IPI, 8, 8);
}

void printByte(int x) {
    int i;
    for (i = 7; i >= 0; i--)
        printf("%d", (x >> i) & 1);
}

int main(void) {
    int plaintext[] = {0x01, 0x02, 0x04};
    int ciphertext[3], recovered[3];
    int counter, i;

    keygen(0x1FD);  /* Key: 0111111101 */

    /* CTR encryption */
    for (i = 0; i < 3; i++) {
        counter = i;  /* Starts at 00000000 */
        ciphertext[i] = plaintext[i] ^ encrypt(counter);
    }

    printf("Ciphertext: ");
    for (i = 0; i < 3; i++) {
        printByte(ciphertext[i]);
        printf(" ");
    }

    /* CTR decryption: use the same counters */
    for (i = 0; i < 3; i++) {
        counter = i;
        recovered[i] = ciphertext[i] ^ encrypt(counter);
    }

    printf("\nDecrypted plaintext: ");
    for (i = 0; i < 3; i++) {
        printByte(recovered[i]);
        printf(" ");
    }

    printf("\n");
    return 0;
}
```

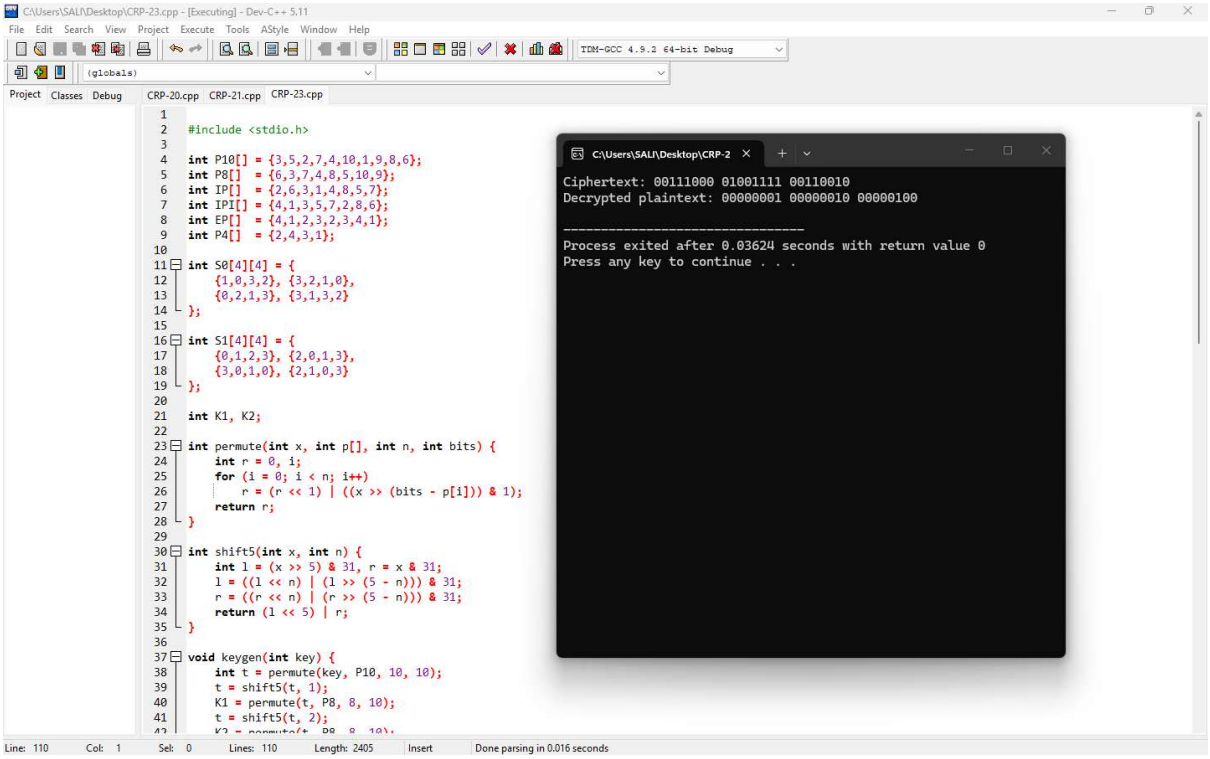
SAMPLE INPUT

The values are defined inside the program.

```
Key:        0111111101
Counter:    00000000
Plaintext: 00000001 00000010 00000100
```

SAMPLE OUTPUT

```
Ciphertext: 00111000 01001111 00110010
Decrypted plaintext: 00000001 00000010 00000100
```



RESULT

The C program successfully performs encryption and decryption using S-DES in Counter (CTR) mode. The generated ciphertext matches the given test data, and decryption restores the original plaintext.

Note: S-DES is a simplified educational cipher and should not be used to protect real-world data.